

Hands-on Activity 6.1 Variables and Data Type in JavaScript	
Course Code: CPE025 B	Program: BSCPE
Course Title: Software Design	Date Performed: 03/02/2026
Section: CPE32S4	Date Submitted: 03/02/2026
Name: Bautista, Jhon Hendricks	Instructor: Engr. Neal Barton James Matira
1. Discussion	
Dialog windows offer a functional way to communicate with a program, allowing you to retrieve data and account for user decisions that influence the software's behavior. While they may not be the most elegant interface, they are highly effective for basic interaction and testing logic. Control Flow and Decision Making Control flow statements are essential for building flexible and reactive applications. They allow you to handle user input, data fetched from the web, or the results of various operations. if...else: This is the most universal construction for making decisions based on conditions. Switch Statement: A convenient alternative to multiple if statements when you have many fixed options to check. Conditional (Ternary) Operator: A concise way to write simple if...else logic in a single line. Loops and Iteration Loops are indispensable for writing efficient programs. They allow you to repeat actions, walk through array elements, or iterate through object fields, making it possible to create solutions that would be impossible to code manually. Several mechanisms are available to handle repetition: while: Repeats a block of code as long as a specified condition remains true. for: The standard loop for iterating a specific number of times. for...in / for...of: Specialized versions of the for loop used to iterate through object properties or array elements. Skillfully using these basic structures provides enough power to create advanced and highly functional programs.	
2. Materials and Equipment	
Computer Integrated Development Environment Internet	
3. Procedure	

3.2.5 LAB Interaction

Estimated time

15-30 minutes

Level of difficulty

Easy

Objectives

Familiarize the student with:

- interacting with the user (dialog boxes – alert, confirm, prompt).

Scenario

Let's go back to our contact list. After some recent tweaks (i.e. using an array and objects) it is actually starting to look like a list. However, one quite serious problem remained. Changing data in the list, such as adding a new contact, has to be provided for right away in the program code. What if we want to give the user the ability to enter the data of the added contact while the program is running? Modify the program to add, at the end of the list, not the contact, which is given in the code, but the one which the user will give during the program run. Use the `prompt` method to do this. At the end, display the first and the last contact from the list.

4.0.9 LAB Conditional execution

Estimated time

30-60 minutes

Level of difficulty

Easy/Medium

Objectives

Familiarize the student with:

- conditional execution (what is conditional execution, the if-else statement, the conditional operator, the switch-case statement)

Scenario

During the last modification of the program with the contact list, we allow the user to add a new contact with the data entered by the user while the program is executing. Let's go a step further – try to modify the program so that the user has a choice of what they want to do with the list. They will have a choice of:

- showing the first contact (first)
- showing the last contact (last)
- adding a new contact (new)

When adding a new contact, check if the user has entered all the necessary data. If at least one of the three values is missing (name, phone, or email) don't add the contact.

4.1.12 LAB Loops

Estimated time

30-60 minutes

Level of difficulty

Easy/Medium

Objectives

Familiarize the student with:

- loops (what are loops, the while loop, the do-while loop, the for loop, the for-of loop, the for-in loop, the break and continue statements)

Scenario

We can improve our contact list program a bit by using loops. You can now try to display not only the first or last contact, but all contacts in the list, regardless of their number.

Additionally, try to enclose the whole program in a loop so that the user is repeatedly asked what they want to do. The user can choose to:

- display the first contact (first)
- display the last contact (last)
- display all contacts (all)
- add a new contact (new)
- exit the program (quit)

After executing the selected action, the program will give the opportunity to choose again. The program should end the actions only after the user gives a specified command, for example `quit`.

4. Output

• 3.2.5 LAB Interaction

```
let contacts = [{
  name: "Maxwell Wright",
  phone: "(0191) 719 6495",
  email: "Curabitur.egestas.nunc@nonummyac.co.uk"
}, {
  name: "Raja Villarreal",
  phone: "0866 398 2895",
  email: "posuere.vulputate@sed.com"
}, {
  name: "Helen Richards",
  phone: "0800 1111",
  email: "libero@convallis.edu"
}];

// get new information
let newName = prompt("Enter your Fullname");
let newPhone = prompt("Enter your Contact Number");
let newEmail = prompt("Enter your Email");

let newContact = {
  name: newName,
  phone: newPhone,
  email: newEmail
};

// add to the back
contacts.push(newContact);

let last = contacts.length - 1;

console.log(`${contacts[0].name} / ${contacts[0].phone} / ${contacts[0].email}`);
console.log(`${contacts[last].name} / ${contacts[last].phone} / ${contacts[last].email}`);
```

Figure 1. New Program

In this new program for us to be able to get new data I used the prompt function to get an input field for the user to put the necessary data and then call these variables in an object. This object is then pushed into the array using the push function. Lastly is display the updated array with the first and last item on the array.

127.0.0.1:5500 says

Enter your Fullname

Jhon Hendricks Bautista

OK Cancel

Figure 2. Prompt for Name

127.0.0.1:5500 says

Enter your Contact Number

1234567890

OK Cancel

Figure 3. Prompt for Contact Number

127.0.0.1:5500 says

Enter your Email

qjhtbautista@tip.edu.ph

OK Cancel

Figure 4. Prompt for Email

```
Maxwell Wright / (0191) 719 6495 / 3.2.5 LAB.js:31  
Curabitur.egestas.nunc@nonummyac.co.uk  
Jhon Hendricks Bautista / 123456789 / 3.2.5 LAB.js:32  
qjhtbautista@tip.edu.ph  
> ctrl i to turn on code suggestions. Don't show again NEW
```

Figure 5. Output of the Console

The last shows the new contact since I pushed the new data at the end of the array.

- 4.0.9 LAB Conditional Execution

```
let options = prompt("Choose an option: first | last | new");
```

Figure 6. Creating Option Prompt

In figure 6 I used the prompt function to get an input field for the user wherein it shows the options they can choose.

```
✓ if (options === "first") {  
    let firstContact = contacts[0];  
    console.log("First Contact:");  
    console.log("Name:", firstContact.name);  
    console.log("Phone:", firstContact.phone);  
    console.log("Email:", firstContact.email);  
✓ } else if (options === "last") {  
    let lastContact = contacts[contacts.length - 1];  
    console.log("Last Contact:");  
    console.log("Name:", lastContact.name);  
    console.log("Phone:", lastContact.phone);  
    console.log("Email:", lastContact.email);  
✓ } else if (options === "new") {  
    let newName = prompt("Enter name:");  
    let newPhone = prompt("Enter phone:");  
    let newEmail = prompt("Enter email:");  
✓ if (newName && newPhone && newEmail) {  
✓     contacts.push({  
        name: newName,  
        phone: newPhone,  
        email: newEmail  
    });  
    console.log("New contact added successfully!");  
    console.log("Updated Contact List:", contacts);
```

Figure 7. Options

In figure 7 I created else if's in order to simulate the option that users can use and then in each I put the proper logic which corresponds to their option. The first is calling the first item on the array, last is showing the last item while the new is for adding new contacts on the array.

```

    } else {
        console.log("All fields are required | Contact Not Added"); // error catching
    }

} else {
    console.log("Invalid Input."); // error catching
};

```

Figure 8. Error Catching

In figure 8 since the problem requires the fields must be filled before pushing these else statements is for catching the errors that a user can make. The first one is nested on the first if for asking the new contact information while the second else statement is for prompting the user when there is an invalid option input.



First Contact: 4.0.9 LAB.js:20
 Name: Maxwell Wright 4.0.9 LAB.js:21
 Phone: (0191) 719 6495 4.0.9 LAB.js:22
 Email: Curabitur.egestas.nunc@nonummyac.co.uk 4.0.9 LAB.js:23
 > `ctrl` `i` to turn on code suggestions. *Don't show again* NEW

Figure 9. First Option



Last Contact: 4.0.9 LAB.js:28
 Name: Helen Richards 4.0.9 LAB.js:29
 Phone: 0800 1111 4.0.9 LAB.js:30
 Email: libero@convallis.edu 4.0.9 LAB.js:31
 > `ctrl` `i` to turn on code suggestions. *Don't show again* NEW

Figure 10. Last Option

127.0.0.1:5500 says

Enter name:

Jhon

OK

Cancel

127.0.0.1:5500 says

Enter phone:

123

OK

Cancel

127.0.0.1:5500 says

Enter email:

qjhtbautista@tip.edu.ph

OK

Cancel

Figure 11. New Option


```
New contact added successfully! 4.0.9 LAB.js:47
Updated Contact List: 4.0.9 LAB.js:48
  ▼ (4) [{...}, {...}, {...}, {...}] ⓘ
    ▶ 0: {name: 'Maxwell Wright', phone: '(0191) 719 6495', email: 'Curabitor
    ▶ 1: {name: 'Raja Villarreal', phone: '0866 398 2895', email: 'posuere.vu
    ▶ 2: {name: 'Helen Richards', phone: '0800 1111', email: 'libero@convalli
    ▶ 3: {name: 'jhon', phone: '123', email: 'qjhtbautista@tip.edu.ph'}
      length: 4
    ▶ [[Prototype]]: Array(0)
  <
> ctrl i to turn on code suggestions. Don't show again NEW
```

Figure 12. New Option Successful

```
Invalid Input. 4.0.9 LAB.js:55
> ctrl i to turn on code suggestions. Don't show again NEW
```

Figure 13. Invalid Input

```
All fields are required | Contact Not Added 4.0.9 LAB.js:51
> ctrl i to turn on code suggestions. Don't show again NEW
```

Figure 14. New Option Incomplete Input

- 4.1.12 LAB

```
while (true) {

    option = prompt("Choose an option: first | last | all | new | quit");

    if (option === "first") {

        let firstContact = contacts[0];
        console.log("First Contact:");
        console.log(firstContact.name, firstContact.phone, firstContact.email);

    } else if (option === "last") {

        let lastContact = contacts[contacts.length - 1];
        console.log("Last Contact:");
        console.log(lastContact.name, lastContact.phone, lastContact.email);

    } else if (option === "all") {

        console.log("All Contacts:");
        for (let i = 0; i < contacts.length; i++) {
            console.log(
                contacts[i].name,
                contacts[i].phone,
                contacts[i].email
            );
        }

    }

}
```

```

} else if (option === "new") {

    let newName = prompt("Enter name:");
    let newPhone = prompt("Enter phone:");
    let newEmail = prompt("Enter email:");

    if (newName && newPhone && newEmail) {

        contacts.push({
            name: newName,
            phone: newPhone,
            email: newEmail
        });

        console.log("New contact added successfully!");
    } else {
        console.log("Error: All fields are required. Contact not added.");
    }

} else if (option === "quit") {

    console.log("Program ended.");
    break;

} else {

    console.log("Invalid option selected.");

}

}

```

Figure 15. Modified Program with Looping

In order to simulate the loop I enclosed the whole previous code in a while true statement. This is so that the program continuously runs and asks the user for an input. For this program, the output is still the same as before but now we have an endless options loop and once the user inputs and option, it will display and the loop continuous.

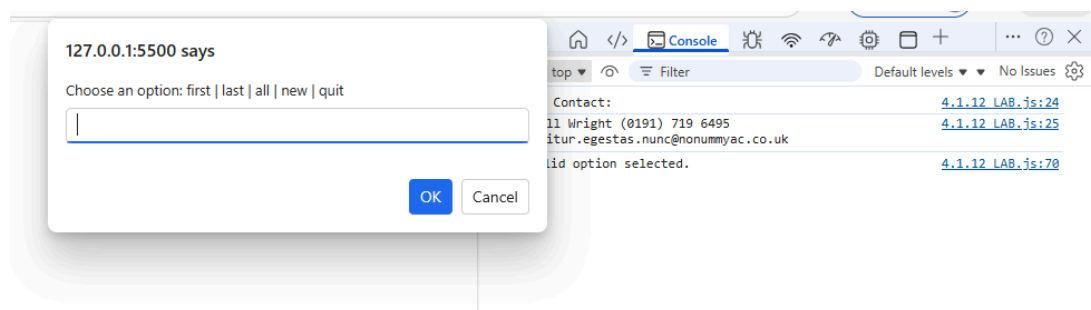


Figure 16. Sample Input

The loop option continues while showing console output of the option.

All Contacts:	4.1.12 LAB.js:35
Maxwell Wright (0191) 719 6495	4.1.12 LAB.js:37
Curabitur.egestas.nunc@nonummyac.co.uk	
Raja Villarreal 0866 398 2895 posuere.vulputate@sed.com	4.1.12 LAB.js:37
Helen Richards 0800 1111 libero@convallis.edu	4.1.12 LAB.js:37
Program ended.	4.1.12 LAB.js:65

Figure 17. All Contact and Program End

In the figure 17 we can see the output of the array is successful through the use of the for loop and then the prompt option field is gone since the user has now typed quit.

5. Supplementary Activity

Scenario

Enhance the contact list application into a robust manager. Instead of just performing one action and stopping, the program should act as a persistent interface.

Starting Data: Use the same initial contacts array from the previous labs:

```
let contacts = [{
  name: "Maxwell Wright",
  phone: "(0191) 719 6495",
  email: "Curabitur.egestas.nunc@nonummyac.co.uk"
}, {
  name: "Raja Villarreal",
  phone: "0866 398 2895",
  email: "posuere.vulputate@sed.com"
}, {
  name: "Helen Richards",
  phone: "0800 1111",
  email: "libero@convallis.edu"
}];
```

Tasks

1. **The Main Menu Loop:** Enclose your entire logic in a loop that keeps the program running until the user explicitly types "quit".
2. **Advanced Menu Options:** The user should be able to choose from the following options via a prompt:
 - a. **show:** Display a specific contact. If the user chooses this, ask for an index number. If the index exists, show that contact; otherwise, show an error message.
 - b. **all:** Display all contacts in the list, one by one.
 - c. **add:** Add a new contact. You must prompt for Name, Phone, and Email separately. *Crucial: Only add the contact if all three fields are filled (not empty).*
 - d. **search:** Ask the user for a Name. Use a loop to check if that name exists in your contacts. If found, display their phone and email. If not found, alert "Contact not found".
3. **Data Integrity:** Ensure that if a user cancels a prompt or enters an invalid option, the program doesn't crash but instead returns to the main menu.

Include Screenshots and Analysis of your Output.

JavaScript

```
let contacts = [{
  name: "Maxwell Wright",
  phone: "(0191) 719 6495",
  email: "Curabitur.egestas.nunc@nonummyac.co.uk"
}, {
  name: "Raja Villarreal",
  phone: "0866 398 2895",
  email: "posuere.vulputate@sed.com"
}, {
  name: "Helen Richards",
  phone: "0800 1111",
  email: "libero@convallis.edu"
}];

let option;

while (true) {

  option = prompt("Choose: show | all | add | search | quit");
  if (option === null) {
    console.log("Returning to main menu");
    continue;
  }

  option = option.toLowerCase();
  if (option === "show") {

    let index = prompt("Select contact index (0-" + (contacts.length
- 1) + "):");
```

```

    if (index === null) {
        continue;
    }

    index = Number(index);

    if (!isNaN(index) && index >= 0 && index < contacts.length) {

        console.log("Contact Found:");
        console.log(
            contacts[index].name,
            contacts[index].phone,
            contacts[index].email
        );

    } else {
        console.log("Invalid index.");
    }

}

else if (option === "all") {
    console.log("All Contacts:");
    for (let i = 0; i < contacts.length; i++) {
        console.log(
            i + ":",
            contacts[i].name,
            contacts[i].phone,
            contacts[i].email
        );
    }

}

else if (option === "add") {
    let newName = prompt("Enter name:");
    let newPhone = prompt("Enter phone:");
    let newEmail = prompt("Enter email:");

    if (newName && newPhone && newEmail) {

        contacts.push({
            name: newName,
            phone: newPhone,
            email: newEmail
        });

        console.log("New contact added successfully!");

    } else {

```

```

        console.log("All fields are required | Contact not added.");
    }
}

else if (option === "search") {

    let searchName = prompt("Enter name to search:");
    if (!searchName) {
        continue;
    }

    let found = false;

    for (let i = 0; i < contacts.length; i++) {
        if (contacts[i].name.toLowerCase() ===
searchName.toLowerCase()) {
            console.log("Contact Found:");
            console.log(
                contacts[i].name,
                contacts[i].phone,
                contacts[i].email
            );
            found = true;
            break;
        }
    }

    if (!found) {
        console.log("Contact not found.");
    }

}

else if (option === "quit") {
    console.log("Program ended.");
    break;
}

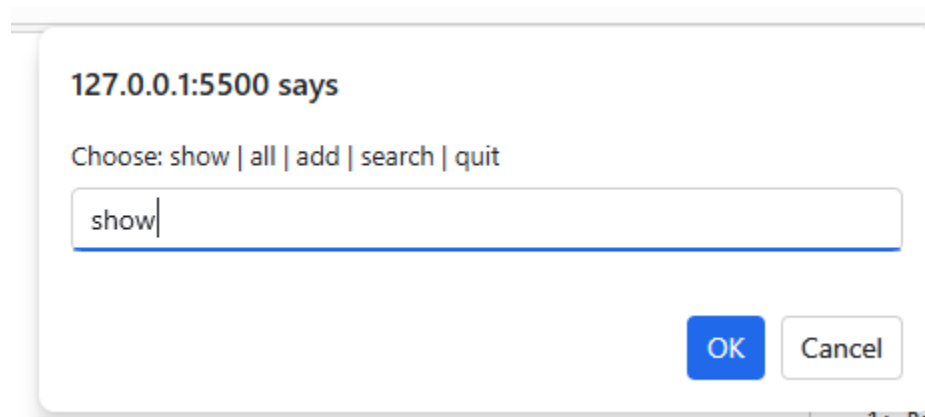
else {
    console.log("Invalid option selected.");
}
}

```

Figure 18. Supplementary Program

For the supplementary activity problem I mainly focus on modifying the program in adding the new option and its functionalities. First is the show option wherein the user can choose an index to output the contact which I performed by using the arrays indexes and shows the user of the length so that they can avoid invalid errors and then once correct the program will call the item based on the input index else it will output the error message since the index is not present on the array. Then another additional functionality

is that the program can now find people through the search option wherein I used the combination of if statements and comparison operators to check if the the input name is present on any of the items on the array. Then it will output the name if it is present and it will output an error message if it is not found. Mainly this program is composed of the main while loop structure and then contains the different if statements and logical and comparison operators for the logic of each options.



A terminal window with a title bar. The text inside reads: "127.0.0.1:5500 says" followed by "Choose: show | all | add | search | quit". Below this is a text input field containing the word "show" with a cursor at the end. At the bottom right are two buttons: "OK" (blue) and "Cancel" (white with blue border).

Figure 19. Show Option



A terminal window with a title bar. The text inside reads: "127.0.0.1:5500 says" followed by "Select contact index (0-3):". Below this is a text input field containing the number "0". At the bottom right are two buttons: "OK" (blue) and "Cancel" (white with blue border).

Contact Found:
Maxwell Wright (0191) 719 6495 Curabitur.egestas.nunc@nonummyac.co.uk

Figure 20. Choosing Correct Index

After choosing the show option once there is a selection of a valid index it will output all the information of the contact on the selected index

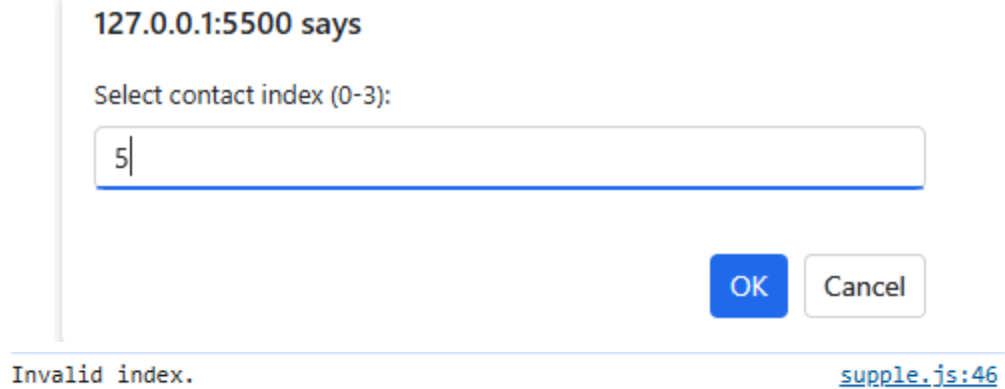


Figure 21. Choosing Incorrect Index

Figure 21 shows the error message once the selected index is invalid

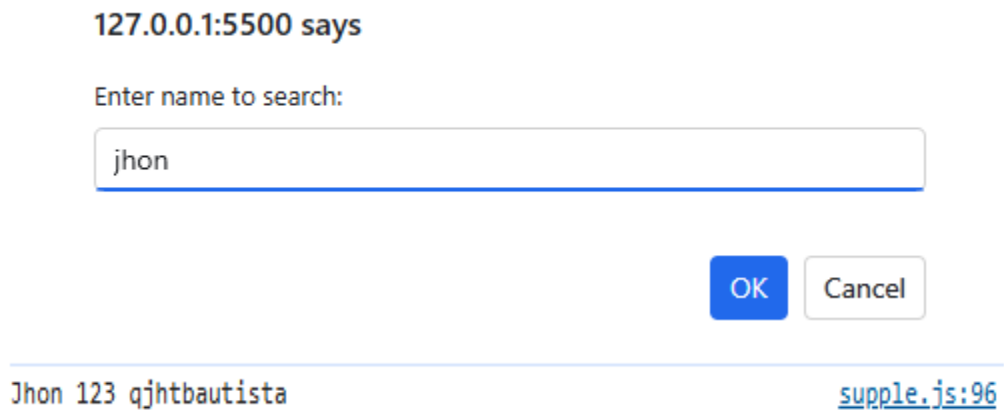


Figure 22. Searching a Name in Array

In figure 22 it shows the search option wherein if we type a valid name it will find the contact which matched with the name and output all of its information.

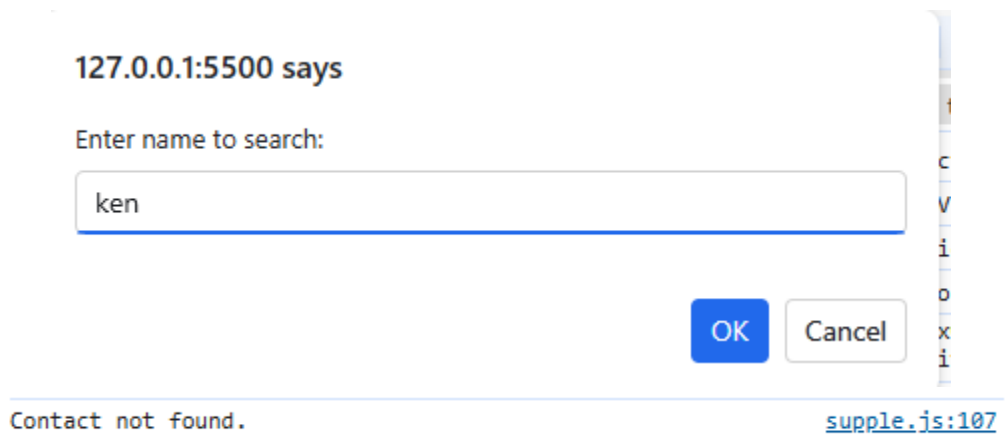


Figure 23. Searching a Name Not in Array

In figure 23 it shows the error message if you typed the wrong name or a name not in the array.

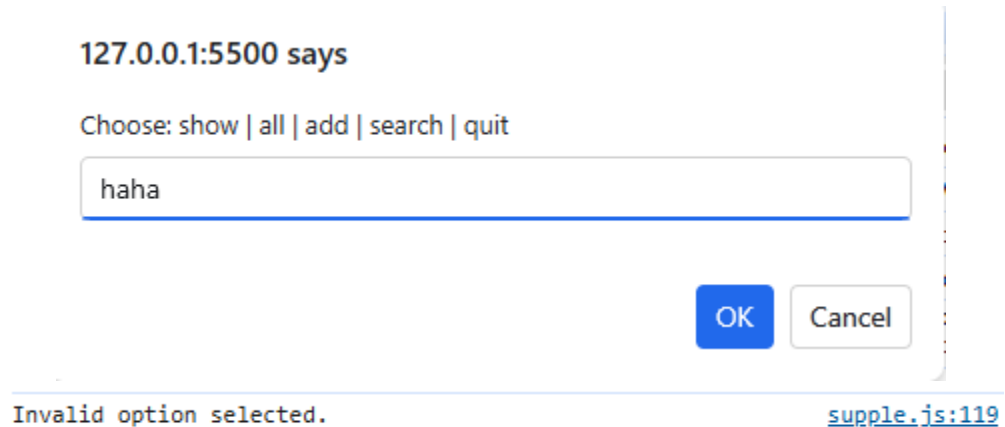


Figure 24. Incorrect Option

The figure 24 shows the error message once the user types an invalid option. Also the program does not close since this still continues the loop of the prompt option because of the proper if else statement.

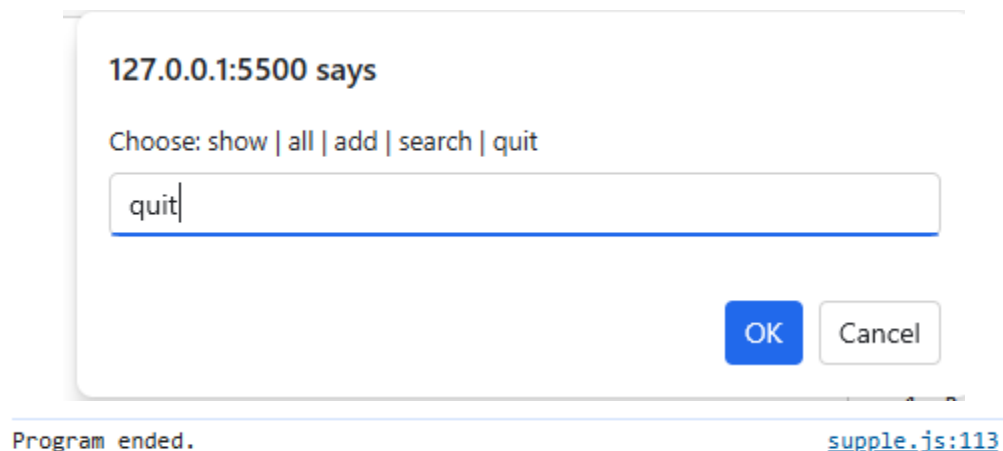


Figure 25. Quit Option Output

Now in figure 25 it shows the output once the user types quit wherein the prompt closes and there is a program ended message

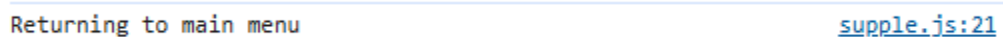


Figure 26. Cancel Option

In figure 26 it shows the console message once the user clicks the cancel button on the prompt.

CONCLUSION

After performing this activity I was able to learn about if statements and flow control in javascript. For this activity I was able to learn the structure of using if statements together with different operators and loops. I was able to utilize my learning in different exercises wherein it require the usage of if statements to perform different options in displaying contact information. Also in this activity I was able to implement the if statements, operators and loops in the supplementary problem in which I was able to make improvements on the initial program and add more features such as finding contacts based on the index

and searching for name of a person which is done with the combination of if statements and operators. Also I was able to create a program that will not break even if there is an error of the input since I learned to utilize the while loop structure and implement proper error catching.